Week 13

ID1063 Introduction to Programming in C

Lecture 23: typedef, Pointers to Structures, Recursion

Announcements

- Endsem **Theory Exam** on **Nov 11th (Wed),** 2pm-5pm
- Endsem Lab exam on **Nov 14th (Sat)**, in batches between 8:30am-6:30pm.
- Only two labs to go, please attend and get your doubts cleared.

Recap: structures so far

- A structure groups related members under one name.
- Members are accessed with the dot operator; assignment copies every member.

```
struct Student {
    int roll;
    char name[20];
    double marks;
};

struct Student st = {42, "Asha", 72.5};
st.marks = 80.0;
struct Student copy = st;      /* copies every member */
```

Passing a structure to a function

- The function receives a **copy** of the structure (pass by value).
- Changes inside the function do not reach the caller's record.

```
void addBonus(struct Student st, double bonus)
{
    st.marks += bonus;          /* changes the copy only */
}

struct Student asha = {42, "Asha", 72.5};
addBonus(asha, 5.0);
printf("%.1f\n", asha.marks);   /* still 72.5 */
```

- Q: How can a function update the caller's record?

Returning a structure

- A function can return a complete record.
- The caller stores the returned copy.

```
struct Student addBonus(struct Student st, double bonus)
{
    st.marks += bonus;
    if (st.marks > 100.0)
        st.marks = 100.0;
    return st;
}

asha = addBonus(asha, 5.0);      /* asha.marks is now 77.5 */
```

- This works, but the whole record is copied twice: once in, once out.

typedef

(naming types)

typedef: a new name for a type

- Syntax: `typedef existing_type NewName;`
- Gives another name to an existing type. No new type is created.

```
typedef unsigned int Count;
typedef double Marks;

Count n = 670;
Marks m = 72.5;              /* m is just a double */
```

- The new name can be used wherever a type is expected: declarations, parameters, return types.

typedef with a structure

- Writing `struct Student` everywhere is long.
- Name the structure type once with `typedef`.

```
typedef struct {
    int roll;
    char name[20];
    double marks;
} Student;

Student asha = {42, "Asha", 72.5};
Student addBonus(Student st, double bonus);
```

- Common style: type names start with a capital letter.

typedef: two ways to write it

- Keep a tag if you also want to write `struct Student`.

Separate definition

```
struct Student {
    int roll;
    char name[20];
    double marks;
};
typedef struct Student Student;
```

Combined

```
typedef struct Student {
    int roll;
    char name[20];
    double marks;
} Student;
```

- In both, `struct Student s;` and `Student s;` declare the same type.
- A tag is needed when a structure contains a pointer to its own type (e.g. linked lists).

Quick check: typedef

- Which of the lines A–D do not compile?

```
typedef struct {
    int x, y;
} Point;

Point p1 = {1, 2};          /* A */
struct Point p2;            /* B */
Point pts[10];              /* C */
Point *pp = &p1;            /* D */
```

Pointers to structures

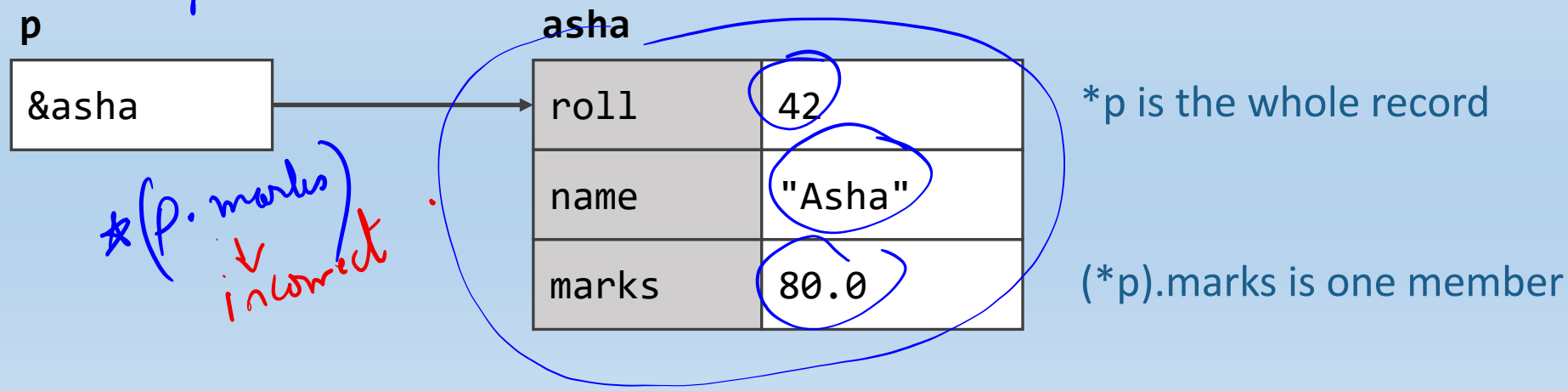
(->, records on the heap, functions)

A pointer to a structure

- A structure variable has an address, like any other variable.
- Store it in a pointer of type `Student *`.

```
Student asha = {42, "Asha", 72.5};
Student *p = &asha;

(*p).marks = 80.0;        /* same as asha.marks = 80.0 */
```



The arrow operator ->

- `p->marks` is shorthand for `(*p).marks`.
- `*p.marks` means `*(p.marks)`: a compile error.

```
Student *p = &asha;

p->marks = 80.0;
printf("%s: %.1f\n", p->name, p->marks);
```

You have	Write	Example
a structure	dot `.`	`asha.marks`
a pointer to a structure	arrow `->`	`p->marks`

Updating through a pointer

- Pass the record's address; the function changes the original.
- Only an address is copied, not the whole record.

```
void addBonus(Student *sp, double bonus)
{
    sp->marks += bonus;
}

addBonus(&asha, 5.0);      /* asha.marks is now 77.5 */
```

By value or by pointer?

	By value	By pointer
Parameter	`Student st`	`Student *sp`
Call	`f(asha)`	`f(&asha)`
What is copied	the whole record	one address
Can change caller's record?	No	Yes
Member access	`st.marks`	`sp->marks`

- Only reading? Use `const Student *`: no copy, and no accidental changes.
- Const keyword disallows assignment to variable involved. Can be used with *any* data type (not just structs)

```
void printStudent(const Student *sp);
```

A structure on the heap

- `malloc(sizeof(Student))` reserves memory for one record.
- Access members with `->`; release the memory with `free`.

```
Student *p = malloc(sizeof(Student));
if (p == NULL)
    return 1;

p->roll = 42;
strcpy(p->name, "Asha");      /* not p->name = "Asha" */
p->marks = 72.5;

/* ... use the record ... */
free(p);
```

A function that creates a record

- Returns the address of a new record on the heap.
- Safe: heap memory outlives the function. The caller must `free` it.

```
Student *makeStudent(int roll, const char *name,
                     double marks)
{
    Student *p = malloc(sizeof(Student));
    if (p == NULL)
        return NULL;      /* memory check */
    p->roll = roll;
    strcpy(p->name, name);      /* assumes it fits */
    p->marks = marks;
    return p;
}
```

An array of records on the heap

- Number of records known only at run time: allocate n of them.
- `cls[i]` is a structure; `cls + i` is a pointer to it.

```
int n;
scanf("%d", &n);
Student *cls = malloc(n * sizeof(Student));
if (cls == NULL)
    return 1;

for (int i = 0; i < n; i++)
    scanf("%d %19s %lf", &cls[i].roll,
          cls[i].name, &cls[i].marks);

printf("%.1f\n", cls[2].marks);
printf("%.1f\n", (cls + 2)->marks);   /* same */
free(cls);
```

Quick check: find the bugs

- Each marked line has a bug. What is it?

```
Student *p = malloc(sizeof(Student *));   /* A */
p.marks = 50.0;                           /* B */
*p.roll = 7;                              /* C */
free(p);
printf("%d\n", p->roll);                  /* D */
```

Recursion

(functions that call themselves)

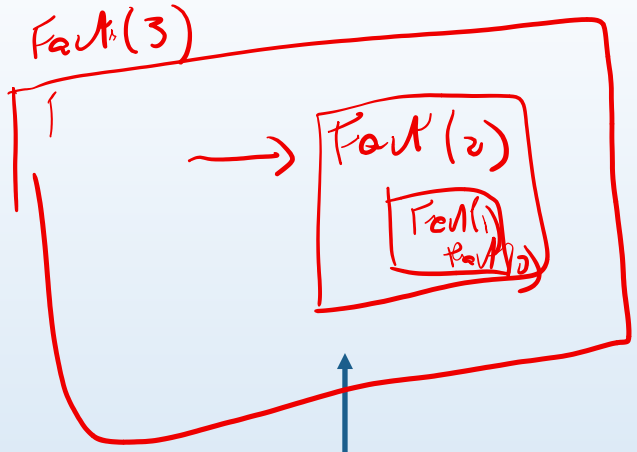
A function that calls itself

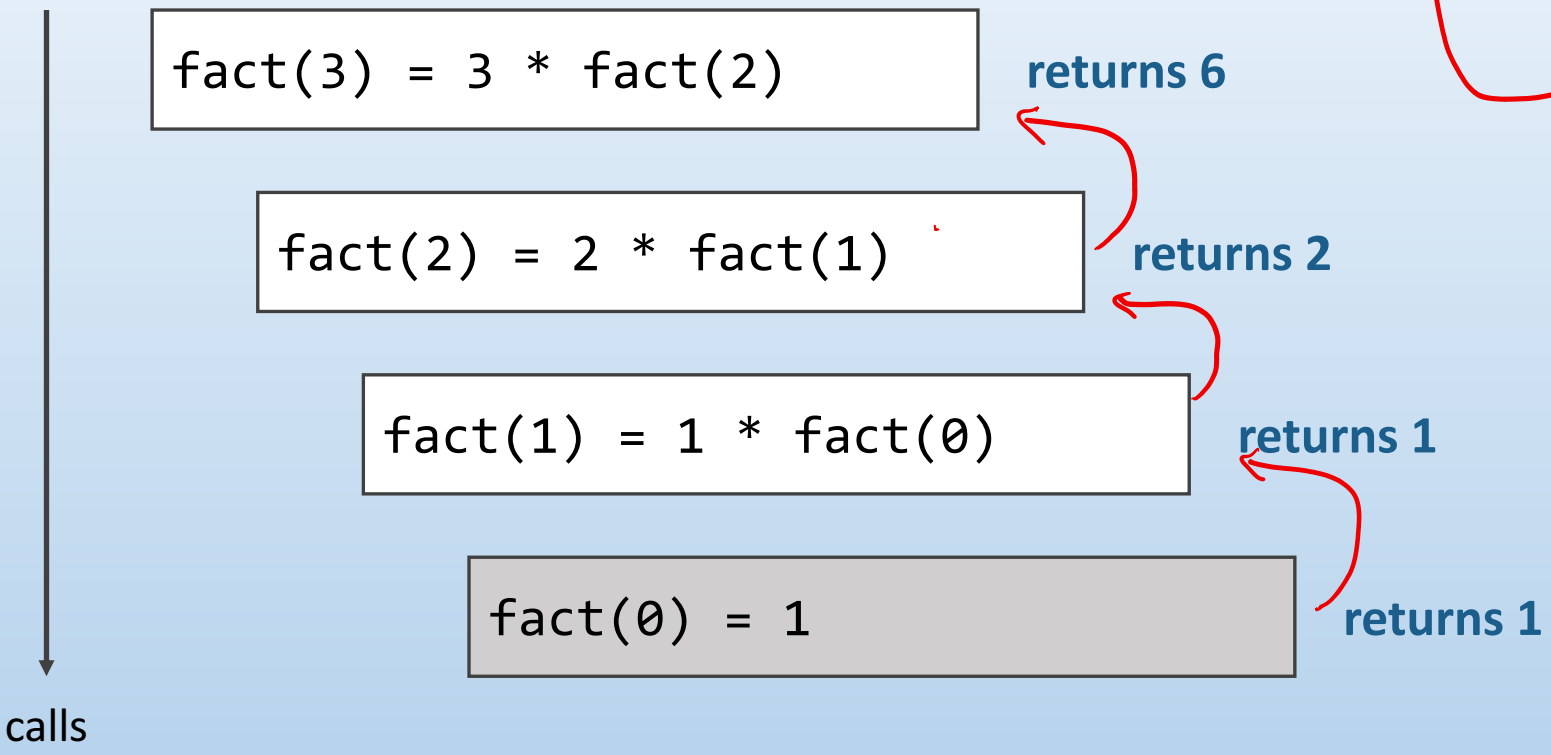
- Solve a problem using a smaller version of the same problem.
- Factorial: $n! = n \times (n - 1)!$ and $0! = 1$

```
int fact(int n)
{
    if (n == 0)                     /* base case */
        return 1;
    return n * fact(n - 1);         /* recursive case */
}
```

- **Base case:** answered directly, with no further calls.
- **Recursive case:** the function calls itself on a smaller input.

Tracing fact(3)





- Each call waits until the call it made returns.
- Each call has its own copy of n, in its own stack frame.

Getting recursion right

- Every recursion needs a **base case**, and each call must move towards it.
- Otherwise calls pile up until stack memory runs out: the program crashes.

```
int fact(int n)
{
    return n * fact(n - 1);      /* no base case! */
}
```

- Q: With the correct version, what happens for `fact(-1)`?
- Many recursive functions can also be written with a loop.

Recursion on an array: Sum of elements

- Sum of n elements = last element + sum of the first n − 1.
- Base case: an empty array has sum 0.

```
int sum(int *a, int n)
{
    if (n == 0)
        return 0;
    return a[n - 1] + sum(a, n - 1);
}

int marks[] = {7, 3, 9, 5};
printf("%d\n", sum(marks, 4));      /* 24 */
```